

Automation Exercise – Test Plan

Created by: Bartłomiej Wilk (TaloHomes404)

Spis treści

1. Test Plan – Introduction	3
2. Scope of Testing	3
3. Test Strategy	4
4. Project Creation and Configuration.....	4
5. Test Cases	8
6. Framework and Test Case Automation	13
7. API Testing	20

1. Test Plan – Introduction

The purpose of this document is to present the process from creating a new project to implementing test automation in a web application. This is the second project in which I want to practically apply my knowledge of the fundamentals of test automation in web applications using Playwright and all the tools used throughout this process.

Main frameworks and tools used in the project:

- **Playwright**
- **Maven**
- **Java**
- **IntelliJ**
- **JUnit 5**
- **Allure Reports**
- **CI/CD in Github Actions**
- **Git and Github**

The main subject of the project will be the Automation Exercise web application, a platform created to practically test and improve automation testing skills. The website is available at:

<https://www.automationexercise.com/>

2. Scope of Testing

In Scope:

- User Authentication Module (Login / Registration)
- Shopping Cart and Checkout Module
- Product Search
- Navigation and verification of UI elements display and functionality
- API Testing

Out of scope:

- Payment Gateways
- Performance Testing
- Security Testing

3. Test Strategy

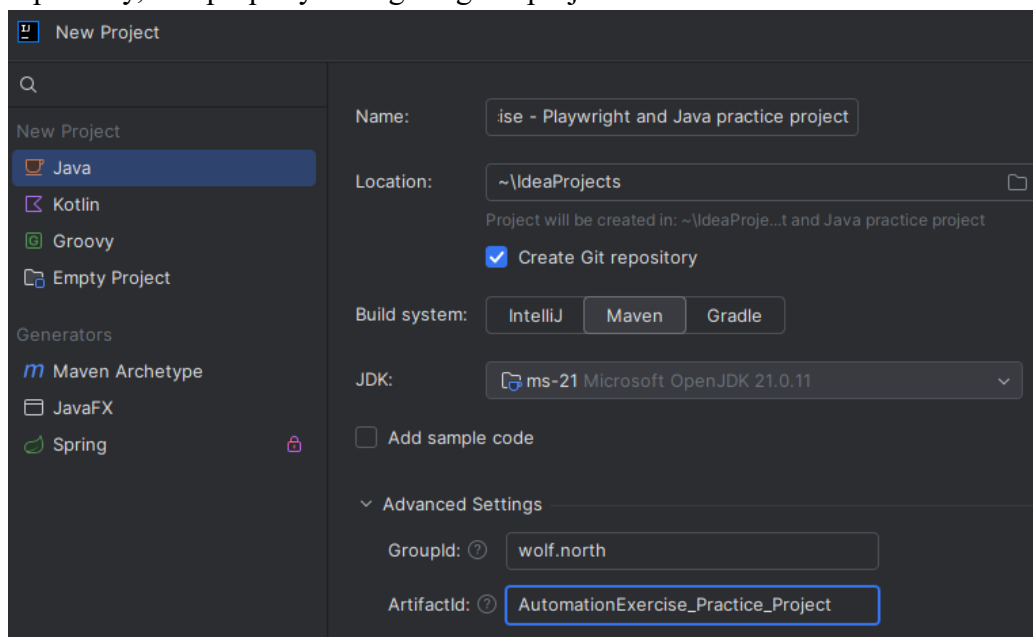
The goal is to ensure high application quality by testing the most critical modules before each production release. By using different types of tests such as smoke tests, end-to-end (E2E) tests, and API tests, we will verify that the application functions correctly and that introduced changes have not caused any of the core modules to work incorrectly.

The key assumption of this testing approach is automation. To reduce the regression testing time of critical modules, test cases will be automated and integrated into the CI/CD pipeline so that they are executed with every major code change.

4. Project Creation and Configuration

- **Project configuration and installation**

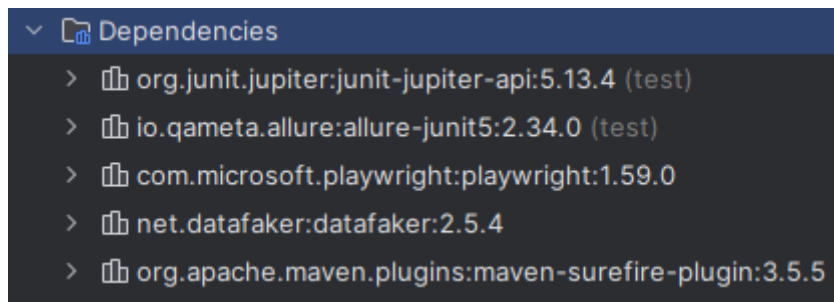
The first step in the configuration process will be creating a new project in the IntelliJ environment and setting up all dependencies in Maven, creating and initializing a Git repository, and properly configuring the project structure.



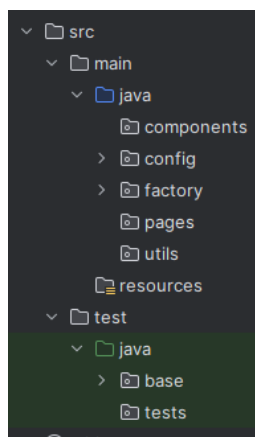
Next, we configure the main Maven file named pom.xml by adding project dependencies from the official repositories for Playwright, JUnit 5, Allure Reports, DataFaker, as well as updating the Surefire plugin. At the same time, we will create and initialize a Git repository and create a new branch where the project configuration will be carried out.

```
Bartek@DESKTOP-07KMHRM MINGW64 ~/IdeaProjects/Automation Exercise - Playwright and Java practice
$ git checkout -b config/init
Switched to a new branch 'config/init'

Bartek@DESKTOP-07KMHRM MINGW64 ~/IdeaProjects/Automation Exercise - Playwright and Java practice
$
```



Next, we create the directory structure and the basic configuration files.



```
public class PlaywrightFactory { 3 usages  Bartłomiej Wilk

    //Init vals for factory
    private static Playwright playwright; 5 usages
    private static Browser browser; 3 usages
    private static BrowserContext context; 3 usages
    private static Page page; 3 usages

    //Init browser and context method
    public static Page initBrowser(){...}

    public static void closeBrowser(){...}
}
```

```

public class ConfigManager { 3 usages new *

    public static final String BASE_URL = no usages
        System.getProperty(key: "env", def: "https://www.automationexercise.com/");

    public static final String BROWSER = 1 usage
        System.getProperty(key: "browser", def: "chromium");

    public static final boolean HEADLESS = 1 usage
        Boolean.parseBoolean(
            System.getProperty(key: "headless", def: "true")
        );
}

```

This is an important step because best practices recommend creating instances in a single place. Not only does this make the configuration easier to manage, but it also allows configuration changes to be made in one location instead of searching through 50 files and manually updating each occurrence.

Next, we created a class in the test/java/base directory named **BaseTest**, which serves as the foundation of our Playwright framework—and is a must-have! This is where the methods responsible for creating the starting point, launching the browser, and executing all actions before and after the tests will be located. A perfect example is the **tearDown** method, which will create screenshots and save test execution information after each test, such as when the test status is **FAIL**.

- **Creating the POM structure**

To ensure proper practices used in automation testing, we will create a simple Page Object Model (POM) structure in our project by creating directories and modules/pages/screens/components that follow the Single Responsibility Principle. This provides better code readability, simplifies maintenance, and enables faster implementation of changes or corrections to UI elements and application flows.

For demonstration purposes, this documentation presents the LoginPage class, which represents the login screen. In our application, this page also contains the initial registration form, which requires entering a username and email address before redirecting the user to the full registration page (which is also represented in the POM structure by the SignupPage class).

```

public class LoginPage {
    private final Page page;

    //== Locators ==//
    //Login Section
    private final Locator loginEmailField;
    private final Locator loginPasswordField;
    private final Locator loginButton;

    //Signup Section
    private final Locator usernameSignupField;
}

```

```

private final Locator emailSignupField;
private final Locator signupButton;

public LoginPage(Page page){
    this.page = page;

    //Login
    this.loginEmailField = page.locator("[data-qa='login-email']");
    this.loginPasswordField = page.locator("[data-qa='login-
password']");
    this.loginButton = page.getByRole(AriaRole.BUTTON, new
Page.GetByRoleOptions().setName("Login"));

    //Signup
    this.usernameSignupField = page.locator("[data-qa='signup-name']");
    this.emailSignupField = page.locator("[data-qa='signup-email']");
    this.signupButton = page.getByRole(AriaRole.BUTTON, new
Page.GetByRoleOptions().setName("Signup"));
}

public void open() { page.navigate(ConfigManager.BASE_URL + "login"); }

public void login(String email, String password){
    loginEmailField.fill(email);
    loginPasswordField.fill(password);
    loginButton.click();
}

public void preRegister(String name, String email){
    usernameSignupField.fill(name);
    emailSignupField.fill(email);
    signupButton.click();
}
}

```

As we can see, the presented solution is designed to separate UI elements and business logic from the test files, where our primary focus is on verifying application behavior rather than the implementation details of individual functions.

5. Test Cases

- User Authentication Module (Login / Registration)

Identifier	TC-REG01 – Successful User Registration
Priority	Highest
Description	Testing the application flow by registering a user with valid data in the system.
Input data	User data generated using the Faker library
Test steps	1. Navigate to the homepage https://automationexercise.com/ 2. Click the "Signup / Login" button. 3. Fill in the fields on the right under "New User Signup!" using the provided test data. 4. Click the "Signup" button. 5. Complete the registration form (Title, Name, Email, Password, Date of Birth). 6. Select the newsletter and special offers checkboxes. 7. Complete the contact information fields (First Name, Last Name, Address, Country, State, City, Zipcode, Mobile Number). 8. Click the "Create Account" button.
Expected result	The user should be successfully registered. After the account is created, the message " ACCOUNT CREATED! " should be displayed. After proceeding further, the user's basic account information should be visible.

Identifier	TC-REG02 – Registering a User with Existing Account Data
Priority	High
Description	Testing the application flow by attempting to register a user whose account already exists in the system.
Input data	Name: Robert Email: robertsmith11@email.com
Test steps	1. Navigate to the homepage https://automationexercise.com/ 2. Click the "Signup / Login" button. 3. Fill in the fields under "New User Signup!" using the provided test data. 4. Click the "Signup" button.
Expected result	The user should not be redirected to the next registration step. The error message " Email Address already exist! " should be displayed.

Identifier	TC-LOG01 – Login with Valid Credentials
Priority	Highest
Description	Testing the application flow for logging in with valid registered credentials.
Input data	Email: robertsmith11@email.com Password: robertsmithpass123
Test steps	1. Navigate to the homepage https://automationexercise.com/ 2. Click the "Signup / Login" button. 3. Enter the provided credentials into the "Login to your account" section. 4. Click the "Login" button.
Expected result	The user should be successfully logged in. The text " Logged in as (username) " should be visible.

Identifier	TC-LOG02 – Login with Invalid Credentials
Priority	High
Description	Validation of the login fields, negative test.
Input data	Email: unknownrichard@tlen.pl Password: testpassword123
Test steps	1. Navigate to the homepage https://automationexercise.com/ 2. Click the "Signup / Login" button. 3. Enter the provided credentials into the "Login to your account" section. 4. Click the "Login" button.
Expected result	The user should not be logged into the system. The message " Your email or password is incorrect! " should be displayed.

Identifier	TC-LOG03 – Successful Logout of a Logged-in User
Priority	Medium
Description	Verifying the correct logout functionality.
Input data	Email: robertsmith11@email.com Password: robertsmithpass123
Test steps	1. Navigate to the homepage https://automationexercise.com/ 2. Click the "Signup / Login" button. 3. Enter the provided credentials into the "Login to your account" section. 4. Click "Login". 5. Click "Logout".
Expected result	The user should be successfully logged out and redirected to the homepage. The " Signup / Login " option should be visible again in the top navigation bar.

- Shopping Cart Module

Identifier	TC-CART01 – Add Two Products to the Cart and Validate Display and Pricing
Priority	Highest
Description	Testing the application flow by adding products to the cart and verifying that they are displayed correctly and calculated properly.
Input data	Guest user
Test steps	1. Navigate to the homepage https://automationexercise.com/ 2. Select "Products" from the top navigation bar. 3. In the All Products section, select the first product (Blue Top) and click "Add to cart". 4. Click "Continue Shopping" in the popup. 5. Add the second product (Men Tshirt) to the cart. 6. Click "View Cart".
Expected result	The cart page should display two products: Men Tshirt and Blue Top . The quantity for each should be 1 , and their prices should be Rs. 500 and Rs. 400 respectively.

Identifier	TC-CART02 – Add Multiple Quantities of a Product and Validate Pricing
Priority	High
Description	Verifying quantity display and price calculations for multiple units of the same product.
Input data	Guest user
Test steps	1. Navigate to the homepage. 2. Open Products. 3. Select Sleeveless Dress and click "View Product". 4. Increase Quantity to 4. 5. Click "Add to cart". 6. Click "View Cart".
Expected result	The cart should contain only Sleeveless Dress , the quantity should be 4 , and the total price should be Rs. 4000 .

Identifier	TC-CART03 – Remove Products from the Cart
Priority	Medium
Description	Verifying that products can be removed from the shopping cart.
Input data	Guest user
Test steps	1. Navigate to the homepage. 2. In Featured Items, add Sleeveless Dress to the cart. 3. Open View Cart. 4. Click the "X" button next to the product.
Expected result	The product should be removed and the message " Cart is empty! Click here to buy products. " should be displayed.

Identifier	TC-CART04 – Add a Product from the Recommended Items Section
Priority	Medium
Description	Testing whether products from the Recommended Items section are added to the cart correctly.
Input data	Guest user
Test steps	1. Navigate to the homepage. 2. Scroll to Recommended Items. 3. Add Summer White Top to the cart. 4. Click "View Cart".
Expected result	The cart should contain Summer White Top with Quantity = 1 and Price = Rs. 400 .

Identifier	TC-CART05 – Add a Product to the Cart from the Product Details Page
Priority	High
Description	Testing that products can be added to the cart from the product details page.
Input data	Guest user
Test steps	1. Navigate to the homepage. 2. In Featured Items, select Stylish Dress and click "View Product". 3. Click "Add to cart". 4. Click "View Cart". W wyświetlonym komunikacie naciśnij podświetlony załącznik „View Cart”
Expected result	The cart should contain Stylish Dress with Quantity = 1 and Price = Rs. 1500 .

Product Search Module

Identifier	TC-SEARCH01 – Search for an Existing Product
Priority	Highest
Description	Testing the product search functionality and verifying that the correct product is displayed.
Input data	Guest user
Test steps	1. Navigate to the homepage. 2. Select Products. 3. Enter "Rose" into the Search Product field and click the search icon.
Expected result	The Searched Products section should display Rose Pink Embroidered Maxi Dress .

Identifier	TC-SEARCH02 – Search for a Non-Existing Product
Priority	Highest
Description	Negative test for searching a non-existing product.
Input data	Guest user
Test steps	1. Navigate to the homepage. 2. Open Products. 3. Enter "Kłapa gaźnika matiz 0.8 e" into the search field and click the search icon.
Expected result	The search results page should be displayed, but no products should be found.

- Checkout Module

Identifier	TC-CHECKOUT01 – Checkout as a Logged-in User
Priority	Highest
Description	Testing whether a registered and logged-in user can successfully complete the checkout process.
Input data	Email: robertsmith11@email.com Password: robertsmithpass123 Name on Card: Robert Smith Card Number: 5513 3110 0011 1005 CVV/CVC: 777 Expiry Date: 06 / 31
Test steps	1. Navigate to the homepage. 2. Add Sleeveless Dress to the cart. 3. Open View Cart . 4. Click " Proceed To Checkout ". 5. Click " Place Order ". 6. Enter the payment card information. 7. Click " Pay and Confirm Order ".
Expected result	The message " Your order has been placed successfully! " should be displayed. The user should then be redirected to the order summary page showing " Order placed! " and " Congratulations! Your order has been confirmed! ".

Identifier	TC-CHECKOUT02 – Checkout as a Guest User
Priority	High
Description	Testing whether a guest user can place an order.
Input data	Guest user
Test steps	1. Navigate to the homepage. 2. Add Sleeveless Dress to the cart. 3. Open View Cart . 4. Click " Proceed To Checkout ".
Expected result	The user should be prompted to log in or register using the " Register / Login " option.

Identifier	TC-CHECKOUT03 – Checkout as a Newly Registered User
Priority	Highest
Description	Testing the registration flow required before completing checkout using valid registration data.
Input data	User data generated using the Faker library.
Test steps	1. Navigate to the homepage. 2. Add Sleeveless Dress to the cart. 3. Open View Cart . 4. Click " Proceed To Checkout ". 5. Click the " Register / Login " link. 6. Enter the generated name and email address. 7. Complete the registration process.

	8. Click "Continue" . 9. Open Cart from the top navigation bar.
Expected result	The previously added product should still be present in the cart. The user should be able to continue the checkout process, and the address section should display the address entered during registration.

6. Framework and Test Case Automation

- **Configuring the test structure**

In the previous part of the documentation, I described the process of creating and performing the basic configuration of the project before testing. Appropriate test cases verifying the main and most important modules of the application were also created.

The next step was the comprehensive creation of the project structure using the POM pattern.

Page Object Model – It is a design pattern used in test automation. Its purpose is to separate the business logic and user interface elements from the implementation of the tests themselves.

```
//== Locators ==//
//Card details
private final Locator nameOnCard; 2 usages
private final Locator cardNumber; 2 usages
private final Locator CVCNumber; 2 usages
private final Locator expirationRateMonth; 2 usages
private final Locator expirationRateYear; 2 usages

private final Locator confirmOrderButton; 2 usages

public PaymentPage(Page page) { 1 usage  ⚡ Bartłomiej Wilk
    this.page = page;

    this.nameOnCard = page.locator(selector: "[data-qa='name-on-card']");
    this.cardNumber = page.locator(selector: "[data-qa='card-number']");
    this.CVCNumber = page.locator(selector: "[data-qa='cvc']");
    this.expirationRateMonth = page.locator(selector: "[data-qa='expiry-month']");
    this.expirationRateYear = page.locator(selector: "[data-qa='expiry-year']");

    this.confirmOrderButton = page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Pay and Confirm Order"));
}

@Step("Enter payment details (Card)") 1 usage  ⚡ Bartłomiej Wilk
public void enterCardDetails(String name, String number, String cvc, String expirationMonth, String expirationYear){
    nameOnCard.fill(name);
    cardNumber.fill(number);
    CVCNumber.fill(cvc);
    expirationRateMonth.fill(expirationMonth);
    expirationRateYear.fill(expirationYear);
}
```

The presented code is a complete structure corresponding to the payment page in our test subject. The typical structure of POM files consists of initializing UI elements, which in Playwright are called Locators and Selectors. Then, the class constructor creates the page object assignment. According to the **Single Responsibility** principle, our PaymentPage class

contains only the elements located on this page and assigns the declared Locators to the corresponding user interface elements.

- **Creating test classes**

To reflect the test cases in the form of automated tests, we created test classes for the modules covered by the testing scope. To present the process of creating test classes, I will use RegisterTest as an example, which is the class containing test cases for the registration module.

```
@ExtendWith(TestResultWatcher.class)
public class RegisterTest extends BaseTest {
```

In this piece of code, we can see the initialization of the test class. The next step was inheriting from the previously created BaseTest class in order to initialize the important variables used during testing, as well as the methods responsible for creating the starting point of our tests..

Another important element is the annotation placed before the class declaration. It indicates that we also want to extend our class with the TestResultWatcher class, which will be described in detail later in this documentation. In short, it is a class that monitors test execution and returns the test result (such as pass, fail, or skipped).

- **Creating test cases**

Next, using the methods implemented in the POM classes, we transform the test cases from the documentation into their code implementation.

```
@Test
public void shouldRegisterNewUserSuccessfully() {

    HomePage homePage = new HomePage(page);
    LoginPage loginPage = new LoginPage(page);
    SignupPage signupPage = new SignupPage(page);
    Topbar topbar = new Topbar(page);
    Locator accountCreated = page.locator("[data-qa='account-created']");
    Faker faker = new Faker();
    PopupUtils popupUtils = new PopupUtils(page);
    String firstName = faker.name().firstName();
    String lastName = faker.name().lastName();
    String address = faker.address().fullAddress();
    String email = faker.internet().emailAddress();
    String password = faker.credentials().password();
    Gender gender = faker.options().option(Gender.class);
    int day = faker.number().numberBetween(1, 32);
    int month = faker.number().numberBetween(1, 13);
    int year = faker.number().numberBetween(1990, 2005);
    String state = faker.address().state();
    String city = faker.address().city();
    String zipcode = faker.address().zipCode();
    String phoneNumber = faker.phoneNumber().phoneNumber();

    homePage.open();
    popupUtils.acceptCookiesIfPopupIsVisible();
```

```

topbar.clickLogin();
loginPage.preRegisterAndClickSignupButton(firstName, email);
signupPage.selectGender(gender);
signupPage.enterPassword(password);
signupPage.enterDateOfBirth(day, month, year);
signupPage.signupForNewsletter();
signupPage.signupForSpecialOffers();
signupPage.enterAddressInformations(firstName, lastName, address);
signupPage.selectCountry("United States");
signupPage.enterAddressDetails(state, city, zipcode);
signupPage.enterPhoneNumber(phoneNumber);
signupPage.clickCreateAccount();

assertThat(accountCreated).isVisible();
assertThat(accountCreated).containsText("Account Created!");
}

```

One does not have to be a specialist to notice that the initial implementation of this test case is quite lengthy. Our test begins with several lines of code responsible for generating test data and handling the popup window. In short, this test should verify whether the user can successfully register rather than demonstrate how to initialize and fill in seventeen form fields.

This was the first test case that was created, and although it fulfilled its purpose by accurately presenting the registration process, it also contained several bad practices, such as repeatedly creating Page Object instances in every test, directly creating and using Locators inside the test, and reducing readability by executing the registration process in a highly step-by-step manner.

```

@ExtendWith(TestResultWatcher.class)  Bartłomiej Wilk
public class RegisterTest extends BaseTest {

    @BeforeEach  Bartłomiej Wilk
    void navigateToLoginAndRegisterPage() {
        homepage.open();
        popupUtils.acceptCookiesIfPopupIsVisible();
        topbar.clickLogin();
    }

    @DisplayName("TC-REG01 - Poprawna rejestracja użytkownika w systemie")  Bartłomiej Wilk
    @Feature("Register")
    @Severity(SeverityLevel.CRITICAL)
    @Test
    public void shouldRegisterNewUserSuccessfully(){
        User user = UserFactory.createRandomUser();
        Locator successMessage = signupPage.getAccountCreatedText();
        loginPage.preRegisterAndClickSignupButton(user.getFirstName(), user.getEmail());
        signupPage.completeUserRegistration(user);

        assertThat(successMessage).isVisible();
        assertThat(successMessage).containsText(expected: "Account Created!");
    }
}

```

At a later stage of test development, however, the implementation became significantly cleaner and more readable. This is exactly how Playwright tests should look—they should be understandable enough to be read almost like an instruction manual. Repetitive code was moved to a method annotated with `@BeforeEach`, which means it is executed before every test. This refactoring improved test readability, reduced code duplication, and provided a better separation between business logic and technical implementation. As a result, the tests became easier to maintain and more resistant to changes in the user interface.

- **Allure integration – test reports**

Seeing a green color after a successfully executed test is always satisfying. Unfortunately, this alone is often not sufficient to present as a professional test report. This is where the **Allure** library comes in—a useful tool that collects information from our IDE and presents it in a clear and user-friendly format.

The Allure configuration requires marking the test steps with appropriate annotations, as well as annotating the test itself with suitable metadata. Allure provides numerous annotations that are automatically grouped in the generated report. This is an excellent solution when we want to organize different types of tests and application modules within a single report.

```
@DisplayName("TC-SEARCH01 - Wyszukiwanie prawidłowego produktu")
@Feature("Search")
@Severity(SeverityLevel.CRITICAL)
@Test
public void shouldSuccessfullySearchForExistingProduct() {
```

```
@Step("Click \"View Product\" on product with id {productId}") 1 usage new *
public void openViewProductPage(int productId) {
    page.navigate(url: ConfigManager.BASE_URL + "product_details/" + productId);
}
```

- **Handling screenshots and Trace Viewer**

Playwright provides several useful built-in tools. One of them is the ability to take screenshots and record every step performed during test execution. This functionality is extremely valuable during test debugging and when identifying and eliminating unstable tests, commonly referred to as **flaky tests**.

The only challenge was determining the point at which the screenshot should be taken. JUnit does not provide a built-in mechanism for checking the execution status of a completed test. Therefore, two custom tools were created: `TestResultWatcher` and `ScreenshotExtension`.


```

public class TestResultWatcher implements TestExecutionExceptionHandler
{
    @Override
    public void handleTestExecutionException(ExtensionContext context,
        Throwable throwable) throws Throwable {

        BaseTest baseTest = (BaseTest)
context.getRequiredTestInstance();

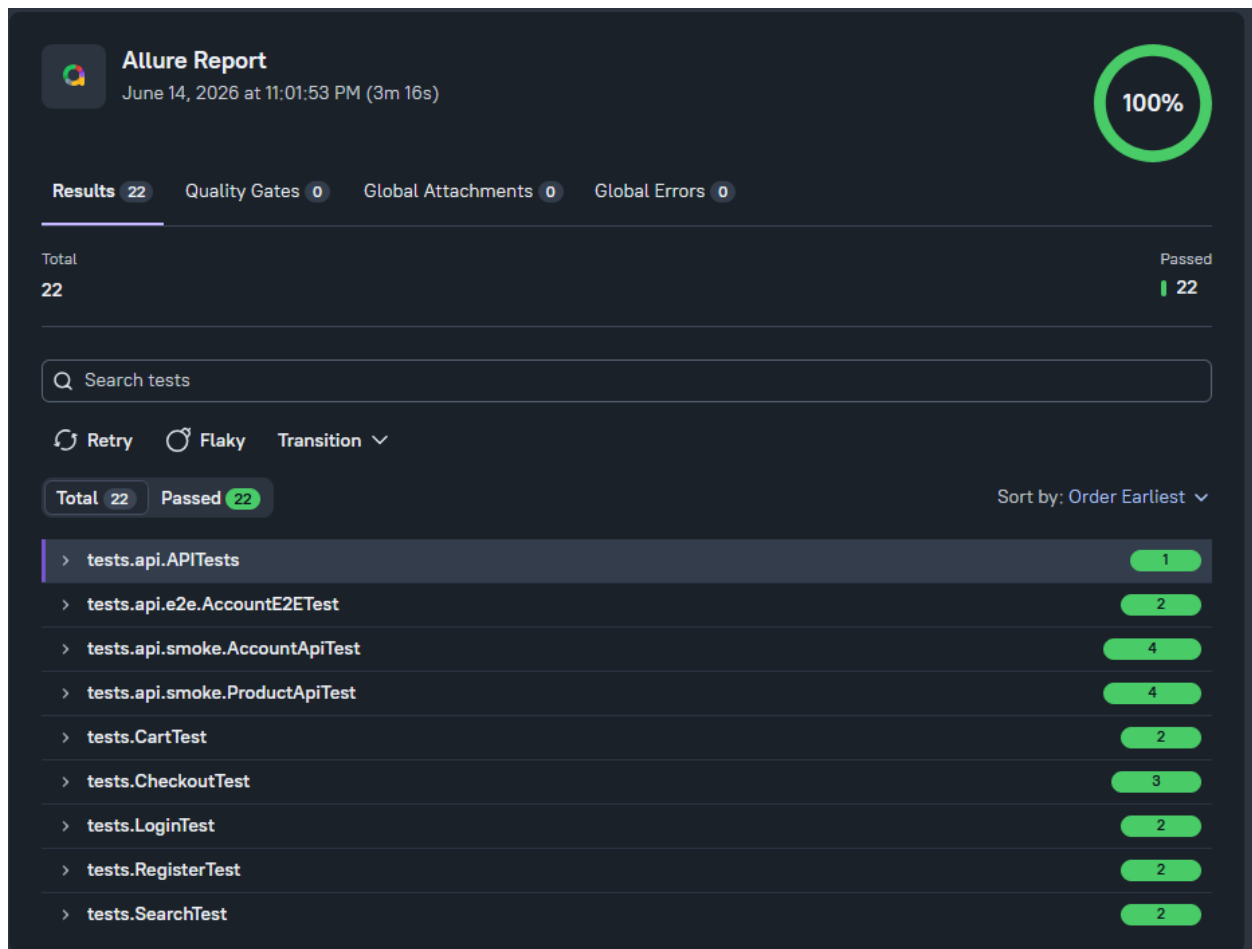
        if (baseTest.getPage() != null) {
            ScreenshotExtension.attachScreenshot(baseTest.getPage());
            ScreenshotExtension.attachCurrentUrl(baseTest.getPage());
        }

        throw throwable;
    }
}

```

This class implements a custom extension using the JUnit 5 `TestExecutionExceptionHandler` interface. It was created to monitor test execution and intercept every exception that causes a test to fail, automatically attaching a screenshot and the current URL to the Allure report.

When using JUnit, this is one of its disadvantages. Another testing framework, TestNG, provides built-in functionality that returns the execution status of a test, making it much easier to determine where to perform actions such as taking screenshots, saving logs, or recording the current page URL.



- **CI/CD implementation in the project**

In one form or another, this concept will eventually be encountered by everyone working in software quality assurance. **CI/CD**, which stands for **Continuous Integration and Continuous Delivery**, was implemented in our project as an important framework component supporting automation.

The tool used, **GitHub Actions**, enables the automation of processes related to building, deploying, and testing the application directly from the GitHub repository. This eliminates the need to manually execute tests after every change committed to the repository and improves the process of detecting and eliminating errors introduced during code changes.

GitHub Actions is an ideal tool for learning the CI/CD process as well as creating project-specific pipelines. The most important step is creating a **Workflow**, which is a file with the .yaml extension containing the instructions that define the automation process for our repository.

```
on:
  push:
    branches: [ master ]
  pull_request:
    branches: [ master ]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Setup Java
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: 21

      - name: Cache Maven packages
        uses: actions/cache@v4
        with:
          path: ~/.m2
          key: ${ runner.os }-maven-${ hashFiles('**/pom.xml') }
          restore-keys: |
            ${ runner.os }-maven-

      - name: Install Playwright browsers
        run: mvn exec:java -e -Dexec.mainClass=com.microsoft.playwright

      - name: Run tests
        run: mvn clean test

      - name: Upload Allure Results
        uses: actions/upload-artifact@v4
        if: always()
```

The prepared pipeline is executed automatically whenever changes are pushed to the main branch of the repository (**push**) and whenever a **pull request** is created or updated. Our pipeline consists of the following stages:

Checkout repository – Downloads the latest version of the source code from the repository.

Setup Java – Configures the Java runtime environment.

Cache Maven packages – Configures dependency caching so that subsequent pipeline executions are faster.

Install Playwright browsers – Installs the required browsers and system dependencies necessary for Playwright to function correctly.

Run Tests – Executes all tests located in the appropriate directories using the **mvn clean test** command.

Upload Allure Results – Archives the generated test reports as Allure results.

7. API Testing

In this section, we will focus on the API testing process, which was also made available in the test subject. We will create an appropriate test structure, assign endpoints, and implement suitable functions that will support the testing process. For API testing, I will use the HTTP client provided by Playwright, which eliminates the need to import separate libraries such as Retrofit or RestAssured into the project.

To verify the correctness of the responses and API calls, I will also use **Postman**, where I will create an appropriate collection, define environment variables, and configure the required request methods together with their mandatory parameters.

- **Scope of testing**
 - Endpoints responsible for retrieving products, creating products, searching for products, and retrieving and adding brands.
 - Endpoints responsible for creating, updating, and deleting user accounts.
- **Out of scope**
 - All performance and security testing.
 - Endpoint load and stress testing will not be performed.

Test Cases – API Smoke Testing

Title	[TC-API01] Retrieve the product list
Required parameters	-
Test steps	1. Send a GET request to the /api/productsList endpoint.
Expected result	Status code: 200 OK The request returns a response containing the list of all products.

Title	[TC-API02] Search for an existing product
Required parameters	Required parameter: search_product
Test steps	1. Send a POST request to the /api/searchProduct endpoint with the required parameter.
Expected result	Status code: 200 OK The request returns a response containing the list of matching products.

Title	[TC-API03] Verify that a correctly registered account exists in the system
Required parameters	Required parameters: email, password
Test steps	1. Send a POST request to the /api/verifyLogin endpoint with the required parameters.
Expected result	Status code: 200 OK The request returns the message "User exists!"

Title	[TC-API04] Retrieve the list of product brands
Required parameters	-
Test steps	1. Send a GET request to the /api/brandsList endpoint.
Expected result	Status code: 200 OK The request returns a response containing the list of all product brands.

Title	[TC-API05] Successfully register a new user
Required parameters	name, email, password, title (Mr./Ms.), birth_date, birth_month, birth_year, firstname, lastname, address1, country, zipcode, state, city, mobile_number – Generated from Faker user model
Test steps	1. Send a POST request to the /api/createAccount endpoint with all required parameters.
Expected result	Status code: 201 Created The request returns the message " User created! "

- **Test Cases – Products and Brand Search**

Title	[TC-API06] Search for products without providing the required parameters
Required parameters	-
Test steps	1. Send a POST request to the /api/searchProduct endpoint.
Expected result	Status code: 400 Bad Request The request returns the error message: "Bad request, search_product parameter is missing in POST request."

- **Test Cases – Account Authorization and Validation**

Title	[TC-API07] Verify account existence without providing the required parameter
Required parameters	Required parameter: password
Test steps	1. Send a POST request to the /api/verifyLogin endpoint with only the required parameter.
Expected result	Status code: 400 Bad Request The request returns the error message: "Bad request, email or password parameter is missing in POST request."

Title	[TC-API08] Update user data without providing any parameters
Required parameters	-
Test steps	1. Send a PUT request to the /api/updateAccount endpoint.
Expected result	Status code: 400 Bad Request The request returns the error message: "Bad request, email parameter is missing in PUT request."

- **End-to-End (E2E) Test Cases**

Title	[TC-E2EAPI-01] Create a user account, verify its existence in the system, and then delete the account
Required parameters	name, email, password, title (Mr./Ms.), birth_date, birth_month, birth_year, firstname, lastname, address1, country, zipcode, state, city, mobile_number – Generated by Faker user model
Test steps	1. Send a POST request to the /api/createAccount endpoint with all required parameters. 2. Send a POST request to the /api/verifyLogin endpoint using the email and password parameters. 3. Send a DELETE request to the /api/deleteAccount endpoint using the email and password parameters.
Expected result	The requests should return the following status codes: 201, 200, and 200, respectively. The user should receive the following messages: "User created!", "User exists!", "Account deleted!"

Title	[TC-E2EAPI-02] Create a new user account, update the existing data, and verify the updated information
Required parameters	name, email, password, title (Mr./Ms.), birth_date, birth_month, birth_year, firstname, lastname, address1, country, zipcode, state, city, mobile_number
Test steps	1. Send a POST request to the /api/createAccount endpoint with all required parameters. 2. Send a PUT request to the /api/updateAccount endpoint using the email and password parameters and set the firstname parameter to "robUpdated". 3. Send a GET request to the /api/getUserDetailByEmail endpoint using the email parameter.
Expected result	The requests should return the following status codes: 201, 200, and 200, respectively. The user should receive the following responses: "User created!" "User updated!" Finally, all user details should be returned, with the firstname field updated to "robUpdated", while all remaining fields remain unchanged.